LAPORAN TUGAS KECIL 2

Voxelization Objek 3D menggunakan *Octree*

Disusun untuk Memenuhi Tugas Kecil 2 Mata Kuliah Strategi Algoritma IF2211 Dosen Pengampu:

Ir. Rila Mandala, M.Eng., Ph.D.

Muhammad Nur Majiid 13524028

Michael James Liman 13524106

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **1**
BAB I **2**
1.1. Deskripsi Masalah 2
BAB II **2**
2.1. Implementasi Program 2
2.2. Algoritma 3
BAB III **5**
3.1. Repository Program 5
3.2. Source Code Program 5
BAB IV **10**
4.1. Test Case 1 10
4.2. Test Case 2 10
4.3. Test Case 3 10
4.4. Test Case 4 10
4.5. Test Case 5 10
4.6. Test Case 6 10
4.7. Tese Case 7 11
BAB V **11**
BAB VI **11**
PERNYATAAN **12**

BAB I
DESKRIPSI MASALAH

1.1. Deskripsi Masalah

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

BAB II
IMPLEMENTASI DAN ALGORITMA

2.1. Implementasi Program

Program *voxelisasi* model 3D ini diimplementasikan menggunakan paradigma prosedural dengan bahasa Go. Penyelesaian persoalan utama dilakukan menggunakan pendekatan algoritma *Divide and Conquer* yang diimplementasikan melalui struktur data *Octree* pada ruang tiga dimensi. Tahap *divide* dilakukan dengan membagi kubus ruang pembungkus model *(Bounding Box*) menjadi delapan sub-kubus (*oktan*) yang berukuran proporsional secara rekursif. Selanjutnya, tahap *conquer* menyelesaikannya dengan mengecek irisan pada setiap sub-kubus terhadap bentuk-bentuk segitiga (*mesh*) penyusun model menggunakan metode pengujian interseksi *Axis-Aligned Bounding Box* (AABB).

Apabila telah sampai pada kedalaman maksimal dari rekursi dan ditemukan irisan, penyelesaian sub-masalah tercapai dengan menandai sub-kubus tersebut sebagai satu buah *voxel* solid, sebaliknya bila ruang terdeteksi sebagai ruang hampa program akan langsung menghentikan penelusuran. Kemudian pada tahap *combine*, seluruh cabang hasil pemeriksaan yang berupa representasi ruang dikumpulkan dan diikat untuk menyatukan utuhnya struktur pohon *Octree* secara keseluruhan. Algoritma utama *Divide and Conquer* untuk penyelesaian masalah ini direpresentasikan secara utuh di dalam subprogram *BuildOctree*. Dibuat juga tipe bentukan dan *subprogram* lain untuk mendukung eksekusi seperti *parseOBJ*, fungsi pengecekan *bounding box*, serta fungsi untuk mengekspor hasil akhir, namun laporan ini difokuskan untuk memaparkan struktur tipe dan subprogram yang relevan pada inti algoritma domain masalah, berikut rancangan dasarnya:

Fungsi	Deskripsi
splitCube8(cube Cube)	Membagi kubus menjadi 8 sub-kubus (oktan) yang sama besar dengan menghitung titik tengah pada setiap sumbu (x, y, z)
calculateCenter(cube Cube)	Menghitung dan mengembalikan titik

	tengah (*center*) dari kubus
IsTriangleInsideCube(triangle Triangle, cube Cube)	Menguji apakah *bounding box* segitiga beririsan dengan sebuah kubus menggunakan AABB *overlap test* pada ketiga sumbu (x, y, z). Mengembalikan true jika beririsan, false jika tidak
BuildOctree(triangles []Triangle, cube Cube, depth int, maxDepth int)	Membangun pohon *Octree* secara rekursif dengan *concurrency*. Menyaring segitiga yang beririsan dengan kubus, membagi kubus menjadi 8 oktan, dan merekursi tiap oktan. Pada depth < 3, 8 rekursi dijalankan secara paralel (*goroutine* + WaitGroup). Pada depth >= 3, dijalankan secara sekuensial.
collectVoxelCenters(node *Node, voxels *[]Vec3)	Menelusuri pohon *Octree* secara rekursif *depth-first*. Mengumpulkan titik tengah (*Center*) dari semua node yang berstatus *voxel* ke dalam array *voxels*

2.2. Algoritma

Algoritma *Divide and Conquer* adalah pendekatan algoritma yang memecah suatu masalah dimensi besar menjadi beberapa sub-masalah yang sejenis dan lebih kecil secara berulang, hingga sub-masalah tersebut cukup sederhana untuk diselesaikan secara langsung (fase *conquer*). Solusi dari kelompok sub-masalah tersebut di akhir akan saling dikombinasikan untuk membentuk solusi kesatuan atas masalah awal. Algoritma ini bersifat sangat adaptif karena mampu mengefisiensikan kinerja komputasi dan ruang memori dengan cara langsung menyingkirkan sub-masalah yang dipastikan tidak memiliki solusi valid (*pruning*). Hal ini sangat cocok diaplikasikan pada kasus penelusuran letak ruangan solid pada bangun maya 3 dimensi. Berikut adalah langkah-langkah jalannya penyelesaian algoritma *voxelisasi* model 3D dengan instruksi menggunakan pendekatan *Divide and Conquer*:

1. Persiapan Data dan Bounding Box Utama, Algoritma diawali dengan membaca data struktur jaring (*mesh*) dari file model 3D (OBJ) untuk memisahkan kumpulan *vertex* (titik koordinat) dan kumpulan permukaannya yang berbentuk formasi segitiga. Setelah selesai diuraikan, program membaca titik-titik tersebut untuk mengkalkulasi sebuah *Bounding Box*, yakni satu kotak pembungkus raksasa yang melingkupi keseluruhan tata letak objek 3D. Program juga menyiapkan parameter maksimal kedalaman potongan, misal maxDepth diatur ke angka 7.

2. Pembagian Ruang (Tahap *Divide*), Algoritma membagi ruang dari area Bounding Box tersebut menjadi tepat delapan bingkai sub-kubus (disebut oktan) secara simetris dan proporsional melalui fungsi splitCube8.

3. Evaluasi Irisan Ruang (Tahap *Conquer*), Pada tahapan ini, di masing-masing dari kedelapan sub-kubus anak akan dilakukan pengujian apakah kotak areanya menabrak dan beririsan dengan lempengan segitiga pada data model. Pengecekan irisan tersebut menggunakan metode deteksi *Axis-Aligned Bounding Box* (AABB) overlap test yang secara rinci menguji ketiga lapisan sumbunya.

4. Penentuan Kasus Basis dan Pemangkasan Ruang, Dari hasil irisan di tes AABB untuk masing-masing ke-8 kubus tersebut, program lalu mengevaluasi masing-masing sub-kubus berdasarkan parameter basis. Ini melahirkan dua kasus penyelesaian basis langsung pada algoritma

    a. *Pruning* Ruang Hampa, Jika sebuah sub-kubus sama sekali tidak ada yang menyentuh segitiga manapun dari model awal, maka ruang area ini seratus persen diasumsikan sebagai "ruang kosong". Iterasi siklus terhadap sub-kubus anak ini langsung dipotong/dibuang, nilainya tidak dikembalikan apa-apa (nil).

    b. Tercipta *Voxel Solid* (Basis Kedalaman Maksimum), Jika sub-kubus beririsan solid dengan poligon objek model, dan iterasi sudah tercatat menyelam mencapai titik maksimum (maxDepth), maka rekursi perulangan dicegat dan diberhentikan. Program lalu segera mencatatkan sub-kubus tersebut valid sebagai balok *voxel* dan merangkum titik tengahnya.

5. Rekursi dengan *Concurrency* (Eksekusi Paralel pada Level Dangkal), Jika terdapat sub-kubus yang beririsan dengan poligon model namun kedalaman belum menyentuh maxDepth, maka sub-kubus tersebut akan secara rekursif mengulangi fase awal. Pada level dangkal (depth

< 3), kedelapan pemanggilan rekursif dijalankan secara paralel menggunakan *goroutine*, di mana setiap *goroutine* memproses satu sub-kubus secara independen dan bersamaan, kemudian disinkronisasi dengan WaitGroup untuk memastikan semua selesai sebelum melanjutkan.

6. Rekursi Sekuensial (Pada Level Dalam), Pada level yang lebih dalam (depth >= 3), kedelapan pemanggilan rekursif dijalankan secara sekuensial satu per satu. Hal ini dilakukan karena pada level yang lebih dalam jumlah node sudah sangat banyak namun beban komputasi per node lebih ringan, sehingga overhead pembuatan goroutine (alokasi stack, *scheduling*, sinkronisasi) akan melebihi keuntungan paralelisme.

7. Penggabungan (Tahap *Combine & Ekspor*): Rangkaian belahan oktan-oktan rekursif berstruktur pohon bercabang tersebut dipadukan kembali (*Combine*). Algoritma *tracking* menelusuri tiap dahan pohon *Octree* yang utuh dan mengumpulkan semua pusat lokasi sub-kubus yang sudah berstatus resmi sebagai *voxel*, untuk digabungkan menjadi file model kotak-kotak *voxel* yang sesungguhnya.

8. Algoritma rekursi di atas akan terus menerus mencoba dan menyeleksi ke arah mana kedudukan irisan ruangan poligon menjalar, dengan memanfaatkan eksekusi paralel pada level atas untuk mempercepat proses.

BAB III

SOURCE CODE

3.1. Repository Program

Tautan menuju repository program adalah sebagai berikut:

https://github.com/MichaelJamesL/Tucil3_Stima.git

3.2. *Source Code* Program

3.2.1. Fungsi IsTriangleInsideCube

```
function IsTriangleInsideCube (input triangle : Triangle, input cube :
Cube) -> boolean
{ Diberikan sebuah segitiga dan sebuah kubus,
  menghasilkan true jika bounding box segitiga beririsan dengan kubus,
  false jika tidak beririsan (SAT - Separating Axis Test pada AABB) }

KAMUS LOKAL
  triangleMin, triangleMax : Vec3

ALGORITMA
```

```
  { Hitung bounding box dari segitiga }
  triangleMin.x <- Min(triangle.v1.x, Min(triangle.v2.x, triangle.v3.x))
  triangleMin.y <- Min(triangle.v1.y, Min(triangle.v2.y, triangle.v3.y))
  triangleMin.z <- Min(triangle.v1.z, Min(triangle.v2.z, triangle.v3.z))

  triangleMax.x <- Max(triangle.v1.x, Max(triangle.v2.x, triangle.v3.x))
  triangleMax.y <- Max(triangle.v1.y, Max(triangle.v2.y, triangle.v3.y))
  triangleMax.z <- Max(triangle.v1.z, Max(triangle.v2.z, triangle.v3.z))

  { Tes separasi pada setiap sumbu }
  if (triangleMin.x > cube.Max.x) or (triangleMax.x < cube.Min.x) then
    -> false
  if (triangleMin.y > cube.Max.y) or (triangleMax.y < cube.Min.y) then
    -> false
  if (triangleMin.z > cube.Max.z) or (triangleMax.z < cube.Min.z) then
    -> false

  -> true
```

Fungsi IsTriangleInsideCube mengimplementasikan tahap *Conquer* dengan mengevaluasi apakah sebuah subruang beririsan dengan permukaan ruang model. Fungsi ini menggunakan metode pengujian interseksi Axis-Aligned Bounding Box (AABB) berdasarkan titik sudut minimal dan maksimal dari permukaan penyusun kubus dan jaring segitiga. Program mengecek tumpang tindih nilai koordinat pada ketiga sumbu secara independen. Apabila ditemukan separasi pada salah satu sumbu saja, maka dapat dipastikan segitiga tersebut berada murni di luar sub-ruang kubus dan fungsi mengembalikan nilai false. Sebaliknya, jika terdapat bagian yang tumpang tindih pada seluruh sumbu fungsi mengembalikan nilai true yang berarti ada bagian model pada ruang yang diujikan sehingga area kubus ini ditandai sebagai area relevan untuk ditelusuri.

3.2.2. Fungsi splitCube8

```
function splitCube8 (input cube : Cube) -> array [0..7] of Cube
{ Diberikan sebuah kubus, menghasilkan array berisi 8 sub-kubus
  hasil pembagian kubus menjadi 8 oktan yang sama besar }

KAMUS LOKAL
  mid : Vec3

ALGORITMA
  mid.x <- (cube.Min.x + cube.Max.x) / 2
```

```
  mid.y <- (cube.Min.y + cube.Max.y) / 2
  mid.z <- (cube.Min.z + cube.Max.z) / 2

  -> [
    <cube.Min, mid>,
    <<mid.x, cube.Min.y, cube.Min.z>, <cube.Max.x, mid.y, mid.z>>,
    <<cube.Min.x, mid.y, cube.Min.z>, <mid.x, cube.Max.y, mid.z>>,
    <<mid.x, mid.y, cube.Min.z>, <cube.Max.x, cube.Max.y, mid.z>>,
    <<cube.Min.x, cube.Min.y, mid.z>, <mid.x, mid.y, cube.Max.z>>,
    <<mid.x, cube.Min.y, mid.z>, <cube.Max.x, mid.y, cube.Max.z>>,
    <<cube.Min.x, mid.y, mid.z>, <mid.x, cube.Max.y, cube.Max.z>>,
    <mid, cube.Max>
  ]
```

Fungsi splitCube8 merupakan implementasi langsung dari pengerjaan tahap *Divide*. Fungsi ini bertugas untuk memecah secara adil sebuah ruang kubus tiga dimensi menjadi delapan oktan berbentuk sub-kubus yang berukuran proporsional sama besar. Pemecahan tersebut dilakukan dengan mencari titik potong penengah pada sepanjang jarak antara batas koordinat minimum dan batas koordinat maksimum di tiap-tiap sumbu ruang. Hasil pembelahan ruang tersebut kemudian direpresentasikan ulang dan dikembalikan menjadi struktur memori array yang memuat delapan sub-kubus baru untuk dilanjutkan pada pemeriksaan iterasi selanjutnya.

3.2.3. Fungsi BuildOctree

```
function BuildOctree (input triangles : array of Triangle, input cube :
Cube,input depth : integer, input maxDepth : integer) -> pointer to Node

{ Diberikan array segitiga, kubus batas, kedalaman saat ini, dan kedalaman
maksimum,
  menghasilkan pointer ke Node root dari octree.
  Fungsi ini bersifat REKURSIF dengan CONCURRENCY.
  Basis: depth >= maxDepth, node ditandai sebagai voxel.
  Rekurens: kubus dibagi 8 sub-kubus, masing-masing dibangun octree secara
rekursif.
  Pada level dangkal (depth < 3), 8 sub-kubus diproses secara PARALEL
menggunakan
  goroutine dan WaitGroup. Pada level dalam (depth >= 3), diproses secara
sekuensial
  untuk menghindari overhead goroutine yang berlebihan. }

KAMUS LOKAL
  trianglesInCube : array of Triangle
  subCubes : array [0..7] of Cube
```

```
  node : pointer to Node
  triangle : Triangle
  i : integer
  wg : WaitGroup  { sinkronisasi untuk eksekusi paralel }

ALGORITMA
  { Basis: kedalaman maksimum tercapai }
  if (depth >= maxDepth) then
    { Tandai node ini sebagai voxel yang terisi }
    -> pointer ke Node <IsVoxel: true, Center: calculateCenter(cube),
Children: kosong>

  { Kumpulkan segitiga yang beririsan dengan kubus ini }
  trianglesInCube <- [] { array kosong }
  i traversal [0..length(triangles)-1]
    triangle <- triangles[i]
    if (IsTriangleInsideCube(triangle, cube)) then
      append(trianglesInCube, triangle)

  { Jika tidak ada segitiga dalam kubus ini, kembalikan nil }
  if (length(trianglesInCube) = 0) then
    -> nil

  { Rekurens: bagi kubus menjadi 8 sub-kubus dan bangun octree }
  subCubes <- splitCube8(cube)
  node <- alokasi Node baru

  { Gunakan concurrency hanya di level atas (depth < 3)
    untuk menghindari overhead goroutine yang berlebihan }
  if (depth < 3) then
    { PARALEL: jalankan 8 rekursi secara bersamaan menggunakan goroutine }
    i traversal [0..7]
      wg.Add(1)
      jalankan secara paralel (goroutine) dengan parameter (idx <- i, sc <-
subCubes[i]):
        node.Children[idx] <- BuildOctree(trianglesInCube, sc, depth + 1,
maxDepth)
        wg.Done()
    wg.Wait() { tunggu semua 8 goroutine selesai }
  else
    { SEKUENSIAL: di level dalam, jalankan satu per satu }
    i traversal [0..7]
      node.Children[i] <- BuildOctree(trianglesInCube, subCubes[i], depth +
1, maxDepth)

  -> node
```

Fungsi BuildOctree merupakan inti eksekusi dari keseluruhan perulangan algoritma Divide and Conquer pada program untuk mengubah

poligon menjadi *voxel*. Fungsi ini bekerja merangkai pohon struktur Octree secara rekursif dengan memanfaatkan *concurrency* (eksekusi paralel). Pada awal setiap pemanggilan, fungsi memeriksa syarat perhentian basis, apakah penelusuran sub-kubus yang dipantau sudah mencapai kedalaman rekursi maksimal yang ditetapkan (maxDepth). Jika belum mencapai batas, fungsi menyaring poligon segitiga dari model secara lokal dengan memanggil fungsi IsTriangleInsideCube untuk melihat poligon bagian mana yang masuk di ruangan tersebut. Jika ditemukan bahwa ruangan di sub-kubus ini tidak bersinggungan seratus persen dengan bangun (ruang hampa) maka rekursif akan memotong penelusurannya (*pruning*) dengan membuang sub-kubus ini via inisialisasi node nil. Sebaliknya, jika masih terdapat segitiga yang beririsan dengan sub-kubus tersebut, maka fungsi melanjutkan ke tahap Divide dengan memanggil splitCube8 untuk membagi kubus menjadi 8 sub-kubus yang lebih kecil, lalu secara rekursif memanggil dirinya sendiri pada masing-masing sub-kubus tersebut.

Fungsi ini menerapkan strategi concurrency yang bersifat adaptif berdasarkan kedalaman rekursi:

- Pada level atas (depth < 3): Kedelapan pemanggilan rekursif untuk setiap sub-kubus dijalankan secara paralel menggunakan goroutine. Setiap goroutine memproses satu sub-kubus secara independen dan bersamaan. Mekanisme sinkronisasi WaitGroup digunakan untuk memastikan bahwa semua 8 goroutine telah selesai sebelum node dikembalikan. Pada 3 level teratas, ini berarti hingga 8 + 64 + 512 = 584 goroutine dapat aktif secara bersamaan, memanfaatkan seluruh core CPU yang tersedia.
- Pada level dalam (depth >= 3): Pemanggilan rekursif dilakukan secara sekuensial (satu per satu). Hal ini karena pada level dalam, jumlah node yang harus diproses sudah sangat banyak, sehingga overhead pembuatan goroutine (alokasi stack, *scheduling*, dan sinkronisasi) akan melebihi keuntungan paralelisme itu sendiri.

Pemilihan batas depth < 3 merupakan pertimbangan trade-off antara paralelisme dan overhead, dimana level 0-2 memiliki jumlah node yang relatif

sedikit namun setiap node mengandung beban komputasi yang besar (banyak segitiga yang harus disaring), sehingga cocok untuk diparalelkan. Sebaliknya, level lebih dalam memiliki beban komputasi yang lebih ringan per node karena segitiga sudah tersaring, sehingga overhead goroutine menjadi tidak sebanding. Alur keseluruhan dikumpulkan secara gabungan dalam tahap Combine hingga tercapai titik representasi Voxel sejati berbentuk node penuh.

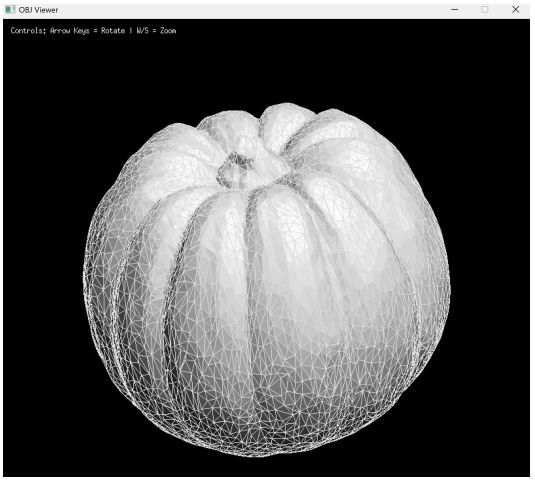
BAB IV
EKSPERIMEN

4.1. Test Case 1

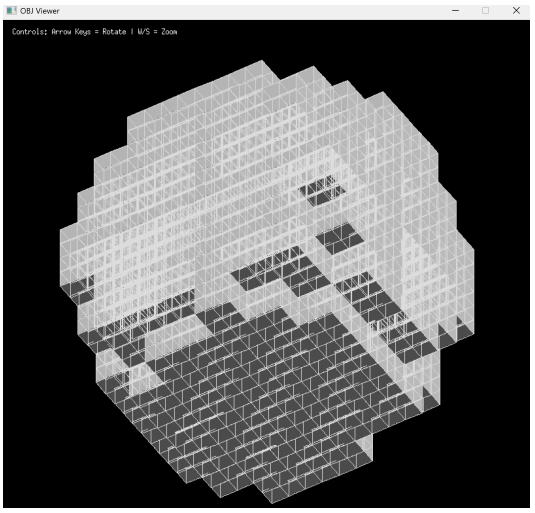
Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 2 Source Obj Filepath: test\in\pumpkin.obj	HASIL VOXELISASI 3D Banyak voxel yang terbentuk : 2496 Banyak vertex yang terbentuk : 19968 Banyak faces yang terbentuk : 14976 Kedalaman octree : 4 Lama waktu program berjalan : 85.6653ms Path file output : C:\Kuliah\Sem 4\stima\tucil\2\voxelized.obj Statistik node octree yang terbentuk: 0 : 1 1 : 8 2 : 56 3 : 312 4 : 2496

	Statistik node yang tidak perlu ditelusuri (pruned): 2 : 8 3 : 136

4.2. Test Case 2

Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 1 OBJ File Path: test\in\pumpkin.obj	

4.3. Test Case 3

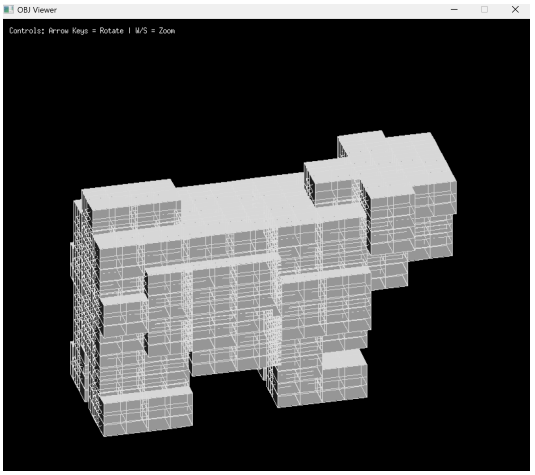
Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 1 OBJ File Path: voxelized.obj	

4.4. Test Case 4

Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 2 Source Obj Filepath: test\in\cow.obj	HASIL VOXELISASI 3D Banyak voxel yang terbentuk : 2016 Banyak vertex yang terbentuk : 16128 Banyak faces yang terbentuk : 12096 Kedalaman octree : 4 Lama waktu program berjalan : 79.7307ms Path file output : C:\Kuliah\Sem 4\stima\tucil\2\voxelized.obj Statistik node octree yang terbentuk: 0 : 1 1 : 8 2 : 50 3 : 252 4 : 2016 Statistik node yang tidak perlu ditelusuri (pruned): 2 : 14 3 : 148

4.5. Test Case 5

Masukan	Keluaran

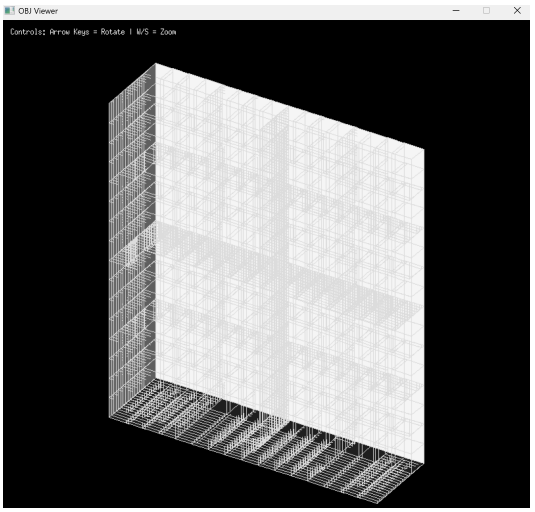
Choose Feature: 1. OBJ Viewer 2. Voxelization 1 OBJ File Path: voxelized.obj	

4.6. Test Case 6

Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 2 Source Obj Filepath: test\in\line.obj	HASIL VOXELISASI 3D Banyak voxel yang terbentuk : 4096 Banyak vertex yang terbentuk : 32768 Banyak faces yang terbentuk : 24576 Kedalaman octree : 4 Lama waktu program berjalan : 92.327ms Path file output : C:\Kuliah\Sem 4\stima\tucil\2\voxelized.obj Statistik node octree yang terbentuk: 0 : 1 1 : 8 2 : 64 3 : 512 4 : 4096

	Statistik node yang tidak perlu ditelusuri (pruned):

4.7. Test Case 7

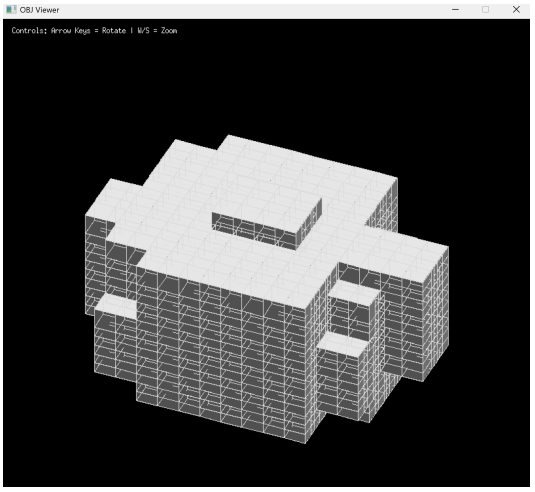
Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 1 OBJ File Path: voxelized.obj	

4.8. Test Case 8

Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 2 Source Obj Filepath: test\in\teapot.obj	HASIL VOXELISASI 3D Banyak voxel yang terbentuk : 2176 Banyak vertex yang terbentuk : 17408 Banyak faces yang terbentuk : 13056 Kedalaman octree : 4 Lama waktu program berjalan : 56.2812ms Path file output :

	C:\Kuliah\Sem 4\stima\tucil\2\voxelized.obj Statistik node octree yang terbentuk: 0 : 1 1 : 8 2 : 60 3 : 272 4 : 2176 Statistik node yang tidak perlu ditelusuri (pruned): 2 : 4 3 : 208

4.9. Test Case 9

Masukan	Keluaran
Choose Feature: 1. OBJ Viewer 2. Voxelization 1 OBJ File Path: voxelized.obj	

BAB V
LAMPIRAN

No	Point	Ya	Tidak

1.	Program berhasil di kompilasi tanpa kesalahan	✔	
2.	Program berhasil dijalankan	✔	
3.	Hasil konversi .obj terdiri dari kubus-kubus kecil	✔	
4.	Program informasi-informasi atau report	✔	
5.	Program dapat menerima masukan .obj, dan mengembalikan output	✔	
6.	[Bonus] Program menggunakan *concurrency*	✔	
7.	[Bonus] Program dapat menampilkan visualisasi .obj	✔	

BAB VI
REFERENSI

Bresenham, J. E. (1965). Algorithm for computer control of a digital plotter. IBM Systems Journal, 4(1), 25-30. https://doi.org/10.1147/sj.41.0025

Catmull, E. (1974). A Subdivision Algorithm for Computer Display of Curved Surfaces. University of Utah (PhD dissertation).

Meagher, D. (1982). Geometric modeling using octree encoding. Computer Graphics and Image Processing, 19(2), 129-147. https://doi.org/10.1016/0146-664X(82)90104-6

Munir, R. (2026). Algoritma Divide and Conquer (Bagian 1) [Materi kuliah]. Institut Teknologi Bandung. Diakses dari : https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf

Phong, B.-T. (1975). Illumination for computer generated pictures. Communications of the ACM, 18(6), 311-317. https://doi.org/10.1145/360825.360839

Pineda, J. (1988). A parallel algorithm for polygon rasterization. ACM SIGGRAPH Computer Graphics, 22(4), 17-20. https://doi.org/10.1145/378456.378457

Wavefront Technologies. (1990s). OBJ File Format Summary.

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Nur Majiid

Michael James Liman